

Strings

Page No.

Date: / /

1. character set / ASCII codes
2. character array
3. String
4. Creating a string

Character set / ASCII codes

A - 65	a - 97	0 - 48
B - 66	b - 98	1 - 49
C - 67	c - 99	2 - 50
:	:	:
:	:	:
:	:	:
Z - 90	3 - 51	9 - 57

(,), . , + , - , * , - -

→ - 10
space - 32
esc - 27

0 - 127 Total 128

7-bits is enough

For storing ASCII

Codes ($2^7 = 128$)

but min^m 1 byte is assigned as it is min^m

available at Unicode
Unicode

2 bytes

16-bits

4x4 bits

→ Hexadecimal form of 4 digits

C 0 3 A

inside heap

Heap

stack

4)

→ 7

Char temp;

✓ temp = 'A'

temp
A
65

✗ temp = 'AB';

✗ temp = A;

✗ temp = "A";

printf("%c", temp) - A

cout << temp; - A

Array OF characters

char x[5];

char x[5] = {'A', 'B', 'C', 'D', 'E'};

x A B C D E
 0 1 2 3 4

char x[] = {'A', 'B', 'C', 'D', 'E'};

char x[] = {65, 66, 67, 68, 69};

char x[5] = {'A', 'B'};

x A B 0 0
 0 1 2 3 4

Strings

char name[10] = {'J', 'o', 'h', 'n', '\0'};

char name[] = {'J', 'o', 'h', 'n', '\0'};

char name[] = "John";

char *n = "John";

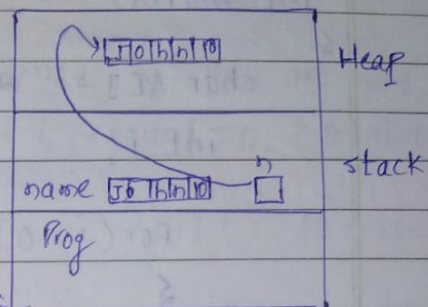
↓
it will be created inside heap

→ printf("%s", name);

→ scanf("%s", name);

↓ *scanf can't read two word string

gets(name); → it can read two word string



Length of a string

s | H | e | l | l | o | | |
 0 1 2 3 4 5 6 7

int main()

{ char *s = "welcome";
 int i;

 for(i=0; s[i] != '\0'; i++)

 {

 {

 printf("Length is %d", i); → 7

 return 0;

}

Changing Case of a string

A	W	E	L	C	O	M	E	\0
	0	1	2	3	4	5	6	7

int main()

{

char A[] = "WELCOME";

int i;

for (i = 0; A[i] != '\0'; i++)

{

A[i] = A[i] + 32;

}

printf("%s", A);

}

A	w	e	l	c	o	m	e	\0
	0	1	2	3	4	5	6	7

↓

int main()

{

char A[] = "wElCome";

int i;

for (i = 0; A[i] != '\0'; i++)

{

if (A[i] >= 65 && A[i] <= 90)

A[i] += 32;

else if (A[i] >= 97 && A[i] <= 122)

A[i] -= 32;

}

Counting vowels and consonants & words

H	O	O		a	r	e		y	O	u		0
0	1	2	3	4	5	6	7	8	9	10	11	

```
int main()
```

```
{
```

```
char A[] = "How are you";
```

```
int i, vcount = 0, ccount = 0;
```

```
for (i = 0; A[i] != '\0'; i++)
```

```
{
```

```
if (A[i] == 'a' || A[i] == 'e' ||
```

```
    A[i] == 'i' || A[i] == 'o' || A[i] == 'u')
```

```
{
```

```
vcount++;
```

```
}
```

```
else if ((A[i] >= 65 && A[i] <= 90) ||
```

```
    (A[i] >= 97 && A[i] <= 122))
```

```
{
```

```
ccount++;
```

```
}
```

```
}
```

Number of words

A		H	O	W		a	r	e				u		0
	0	1	2	3	4	5	6	7	8	9	10	11		

whitespace

```
int main()
```

```
{
```

```
char A[] = "How are you";
```

```
int i, word = 1;
```

```
for (i = 0; A[i] != '\0'; i++)
```

```
{
```

```
if (A[i] == ' ' && A[i-1] != ' ')
```

```
word++;
```

```
}
```

```
printf("%d", word);
```

Validate a string

name A n i ? 5 2 1 \0
 0 1 2 3 4 5 6 7

```
int valid(char *name)
```

```
{
```

```
    int i;
```

```
    for(i=0; name[i]!='\0'; i++)
```

```
{
```

```
        if (!(name[i]>=65 && name[i]<=90)
```

```
            && !(name[i]>=97 && name[i]<=122)
```

```
            && !(name[i]>=48 && name[i]<=57))
```

```
                return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
int main()
```

```
{
```

```
    char *name = "Ani?2r";
```

```
    if (valid(name)) { printf("Valid string");
```

```
    else
```

```
        printf("Invalid string");
```

```
}
```

Reversing a string

A P y t h o n \0
 0 1 2 3 4 5 6

B n o h t y P \0
 0 1 2 3 4 5 6

M-1

```
int main()
```

```
{
```

```
char A[] = "Python";
```

```
char B[7];
```

```
int i;
```

```
for (i = 0; A[i] != '\0'; i++)
```

```
{
```

```
{
```

```
i = i - 1;
```

```
for (j = 0; i >= 0; i--, j++)
```

```
{
```

```
B[j] = A[i];
```

```
{
```

```
B[i] = '\0';
```

```
printf("%s", B);
```

```
{
```

M-2

Mutable ^{means} can be changed

Note:- In recent C, C++

compiler string are

not mutable if you

take pointer string

so take array of

string then it is mutable.

```
int main()
```

```
char A[] = "Python";
```

```
char t;
```

```
int i, j;
```

```
for (j = 0; A[j] != '\0'; j++)
```

```
{
```

```
{ j = j - 1;
```

```
for (i = 0; i < j; i++, j--)
```

```
{
```

```
t = A[i];
```

```
A[i] = A[j];
```

```
A[j] = t;
```

```
printf("%s", A);
```

Comparing string and checking palindrome

A | P | a | i | n | t | e | r | \0
0 1 2 3 4 5 6 7

B | P | a | i | n | t | i | n | g | \0
0 1 2 3 4 5 6 7 8

int main()

char A[] = "Painter";

char B[] = "Painting";

int i, j;

→ For (i=0, j=0; A[i] != '\0' && B[j] != '\0'; i++, j++)

{

if (A[i] != B[j])

break;

}

if (A[i] == B[j]) printf("equal");

else if (A[i] < B[j]) printf("smaller");

else printf("greater");

}

Palindrome

A | m | a | d | a | m | \0
←

M1:-

int main()

{ char A[] = "madam";

char B[];

① Reverse

② Compare

}

M2:- Without taking another string compare in same

Findings Duplicates in a string

1

F	i	n	d	i	n	g
---	---	---	---	---	---	---

 10

0 1 2 3 4 5 6 7

1. Compare with other letters
2. Using HashTable as Counting
3. Using Bits

1. Compare with other letters

↓
same as in Array ADT

Q.2. Using Hash Table as Counting

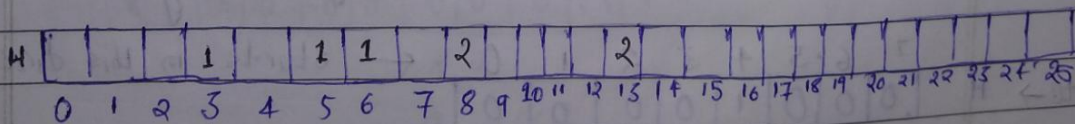
U 102 10^r 110 110 10^r 110 103

A

F	i	n	d	i	n	g	10
0	1	2	3	4	5	6	7 ₂

97 - 122

$$\begin{array}{r} a \\ \downarrow \\ 0 \end{array} \quad \begin{array}{r} 3 \\ \downarrow \\ 25 \end{array}$$



97 → add 97 For Finding alphabet

$$g = 103 - 97 = 6, \quad f = 102 - 97 = 5, \quad h = 110 - 97 = 15$$

$$c = 100 - 97 = 3, \quad d = 100 - 97 = 3$$

```

int main()
{
    char A[] = "Finding";
    int H[26], i;

    For (i = 0; A[i] != '\0'; i++)
    {
        H[A[i] - 'a'] += 1;
    }

    For (i = 0; i < 26; i++)
    {
        IF (H[i] > 1)
        {
            printf("%c", i + 'a');
            printf("%d", H[i]);
        }
    }
}

```

Revise 6

8. Using Bits

Bitwise Operations

1. Left shift <<
2. Bits ORing (Merging)
3. Bits ANDing (Masking)

Byte → H

7	6	5	4	3	2	1	0
0	0	0	1	0	0	0	0
128	64	32	16	8	4	2	1

← starts in this direction

16 = 1 00 00

1. LEFT shift

$H \ll 5$

↓
shift by 5 place

2. Bits ANDing

$a = 10 \rightarrow 1010$

$b = 6 \rightarrow 0110$

$a \& b = 0010 = 2$

3. Bits ORing

$a = 10 \rightarrow 1010$

$b = 6 \rightarrow 0110$

$a | b = 1110 = 14$

Why called Merging

$$\begin{array}{ccccccc}
 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\
 H & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
 & 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1
 \end{array}
 \rightarrow \text{We want to make place 2 as 00}$$

Non Perform Merging

$$\begin{array}{ccccccc}
 A & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
 & 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1
 \end{array}$$

$a = 1$

$a = a \ll 2$

$H = a | A \rightarrow 00010100$

Why called Merging

$$\begin{array}{ccccccc}
 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\
 H & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
 & 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1
 \end{array}$$

$$\begin{array}{ccccccc}
 A & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
 & 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1
 \end{array}$$

How to Find at place

2, 00 or 1 is present

$a = 1 \rightarrow a \ll 2$

Non Perform Merging

$a \& H$

$$\begin{array}{ccccccc}
 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 & 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1
 \end{array}$$

0 is present

Page No. _____
Date: / /

102 105 110 110 105 110 103

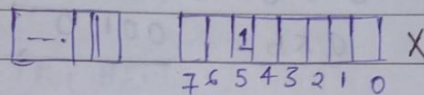
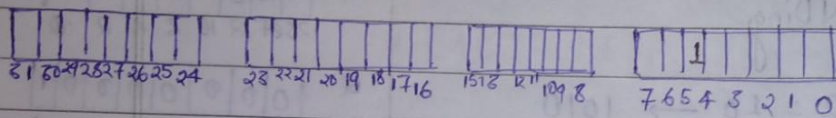
A [f i n d i n g | 0]

0 1 2 3 4 5 6 7

i

$$102 - 97 = 5$$

26 → required
↓ but
32 [least byte]



int main()

{ char A[] = "finding";

log int H=0, x=0;

in some compiler
int H=0, x=0 → A
it takes 4 bytes

for (i=0; A[i] != '\0'; i++)

{ x=1;

→ x = x << (A[i] - 97); // x shifted by 5

→ if (x & H > 0)

{ printf("v.c is duplicate, A[i]");

}

else

→ H = x < H;

}

4. 

int main()

{

char A[] = "decimal";

char B[] = "medical";

int i, H[26] = {0};

For (i = 0; A[i] != '\0'; i++)

{

H[A[i] - 97] += 1;

}

For (i = 0; B[i] != '\0'; i++)

{

H[B[i] - 97] -= 1;

IF (H[A[i] - 97] < 0)

{

printf("not Anagram");

return 0;

}

IF (B[i] == '\0')

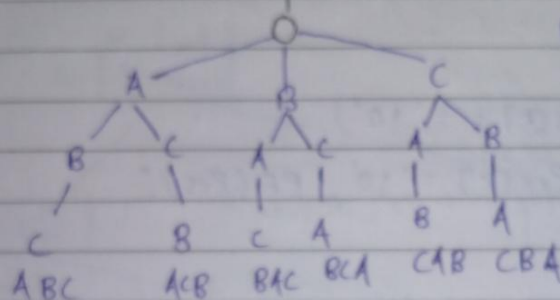
printf("its Anagram");

M3:- He Can use ~~if~~ bitset IF duplicate is not present. (as bitset can't operate for more than 2)

Permutations of a string

S [A B C]

• state space tree



1. ABC

2. ACB

3. BAC

4. BCA

5. CAB

6. CBA

3!

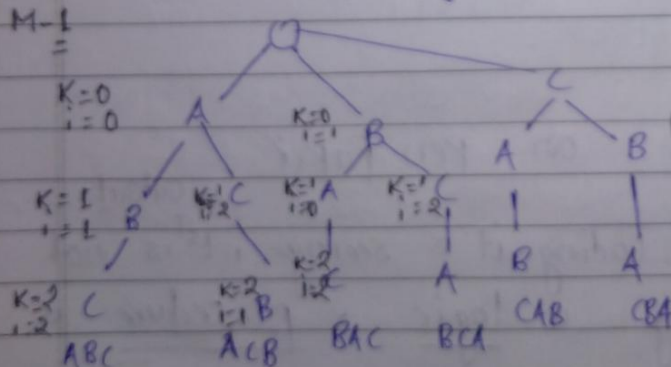
• Backtracking → Going from C to B to A then to A to C to B

• Brute Force → Finding out all possible solution

• Recursion

→ Using recursion we are achieving backtracking
by backtracking we are achieving Brute Force.

M-1
=



To check S [A B C]
which is selected 1 A 5
or not

Flag → A [0 1 1 1]
Actually 0 1 2 3

REG [B C C]
0 1 2 3
k

```

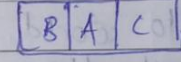
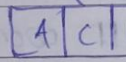
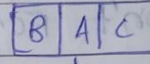
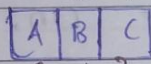
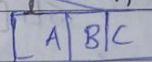
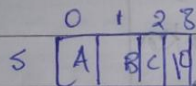
void perm(char s[], int k)
{
    static int A[10] = {0};
    static char Res[10];
    int i;

    if (s[k] == '\0')
    {
        Res[k] = '\0';
        PF(Res);
    }
    else
    {
        for (i = 0; s[i] != '\0'; i++)
        {
            if (A[i] == 0)
            {
                Res[k] = s[i];
                A[i] = 1;
                perm(s, k+1);
                A[i] = 0;
            }
        }
    }
}
    
```

Available for all call

- Do above things on Pen Paper
- Writing a Function, Coding it & seeing its ^{output} is not important, Framing a logic & procedure is more important

IF you work on this by self
then you can find soln
for any other problem


$$\sum_{i=1}^n \int_{\mathbb{R}^d} \dots$$
$$1F(d = h)$$
$$\sum PF(S) =$$

6

e 15e

for ($i = l; i \leq h; i++$)

~~swap(s[l], s[i]).~~
$$\text{Perm}(\sigma_{r, d+1}, h)$$
$$\text{snap}(s[i], s[j])$$

DO Paper-work

17 Remove space From string

→ `char s[50];`

`cin.get(s, 50)`

`cin.get(s, 50)` ^{store in Max=50}

★ `cin << s; John steve` ^{it won't be taken}

M → count = 0

`if (s[i] != ' ')`

`{ str[count++] = s[i];`

`}`

★ Also don't Forget to use `cin.ignore();`

2. Reverse individual words

→ Hello world } Push all words in stack i.e
olleH dlrow } before space

3 check string s1 & s2 are rotation of each other or not

→ s1 = ABCD, s2 = CDAB | s1 = ABCD s2 = ACBD

↓
Yes

↓
No

M → temp = str1 + str1

str1 = "ABCD", str2 = "CDAB"

temp = str1 + str1 = "ABCDABCD"

since str2 is a substring of temp, str1 & str2 are rotation of each other

`if (str1.length() != str2.length())
return false;`

for substring → `str.find(str) < s.length()`

4. Interleaving

M → match F & so

5. Longest P

M1 → Using D

6. Print a

So

str = "Program"

str.substr(0, 3)

↓

str.substr(3, 3)

↓

• S

i

e

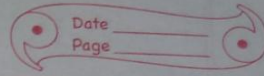
✓

F

8

8

★ ★ ★ ASCII code of space is 32 (Very much Useful in questions)



4. Interleaving of given two string.

A → "AB" B → "CD"

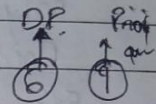
→ Interleaving → ACDB

M → Match first character of C with either A or B & so on.

5. Longest Palindrome in a string
aaaabbaa → aabbaa

M1 → Using DP

M2 → Using searching even/odd length palindromic string



6. Print all subsequences of string.

Some functions (in C++)

- stoi()
- to_string()

Str = "Programming"

s.substr(start, number)

s.substr(3, 4)

s.replace(3, 5, "aa") → ProkKong

no of alphabets

s.erase()

start, no of alphabet

s.substr(3)

programming

s.insert(3, "kk")

0 1 2 3
Hello → Helkklo

s.find("str") → gives starting index

s.rfind("str") → Find occurrence by Rts

s.compare(str)

Teacher's Signature

• S
i
e
r
F
of
E

Q. Reverse words in a string

S may contain leading or trailing spaces or multiple space between two words. The returned string should only have a single space separating the words. Don't include any extra spaces.

Ex:-

Input: s = "the sky is blue"

Output: "blue is sky the"

* Input: s = "--hello-world"

O/P: "world hello"

Brute Force

"--hello-world--"

using two pointer & store result in another string
you did it by self $O(n), O(n)$

↓

Optimize

$O(n), O(1) \rightarrow$ in place

void reverse (string s, int start, int end)

{

while (start < end)

{

swap [s[start++], s[end--]];

}

}

String reversewords (string s)

{

int i = 0, start = 0;

while (s[i] != '\0')

{

reverse (s, start, i);

}

i++;

}

while (i < s.length())

{

use of erase function

{

s.erase (start, i);

re

return

}

Q. Find length of longest substring without repeating characters

E/P: "abcabc" → 3

Output: 3

int

while (i < s.size())

{ if (s[i+1] == s[i])

{ reverse(s, start, i);

start = i+2;

use of erase function

while (i < s.size() && s[i] == s[i+1] && s[i+1] == s[i+2])

{ s.erase(i, 1);

start = i+1;

i++;

{ s.erase(s.size()-1, 1);

reverse(s, 0, s.size()-1);

return s;

Q. Find length of longest palindrome

Eg: "abcccd"

Input: s = "bb"

Output: 7

Output: 2

int f(string s) {

int count = 0;

map<char, int> m;

for (int i = 0; i < s.size(); i++)

{ m[s[i]]++;

if (m[s[i]] % 2 == 0)

{ count += m[s[i]];

for (auto x: m)

{ if (x.second == 1)

{ count++; break;

return count;

Teacher's Signature _____

Q Longest palindromic substring

Input: aaaabbaa
O/P: aabbaa

Input: babad
O/P: bab

Input: cbbd
O/P: bb

Brute Force

↓
Produce all substrings

→ check if palindrome

$O(N^2 \times N)$
↓
Producing $O(N)$ For checking if palindrome all substring

↓
Optimize

(DP)

Input: a a a a b b a a

	0	1	2	3	4	5	6	7
0	1	1						
1		2	2					
2			1	1				
3				1	0			
4					1	1		
5						1	0	
6							1	2
7								1

length a substring → (from index 6 to 7)
length 1 substring → (at index 7)

condⁿ

For Palindrome

only if (a) b c b (a)

(i) $s[start] == s[end]$

(ii) non-boundary substring

dp[i][j-1]

for applying formula

len > 2 should be palindrome

Teacher's Signature _____

To solve more variety of string DP ques

Sub(i,j) → str[i] == str[j] } len >= 3
 && dp[i+1][j-1] == 1

string longestPalindrome (string s) {
 int n = s.size(), start=0, end=0;

int dp[n][n];
 string res;

```
for (int i=0; i<n; i++)
{
  for (int j=0; j<n; j++)
  {
    dp[i][j]=0;
  }
}
```

```
for (int i=0; i<n; i++)
{
  dp[i][i]=1;
  if (i<n-1 && s[i] == s[i+1])
  {
    dp[i][i+1]=1;
    if (i == end - start)
    {
      start = i;
      end = i+1;
    }
  }
}
```

```
for (int j=2; j<n; j++)
{
  for (int i=0; i<n-j; i++)
  {
    if (s[i] == s[i+j] && dp[i+1][i+j-1])
    {
      dp[i][i+j]=1;
      if ((j-1) > (end-start))
      {
        start = i;
        end = i+j;
      }
    }
  }
}
```

res = s.substr(start, end-start+1);
 return res;

Teacher's Signature _____

100%
 Do not
 forget
 to
 return
 the
 string

ng columnwise
 not forget
 we have done
 mistake in going
 row wise

cbdd
 bb

palindrome

palindrome

A-Z → 65-90
 a-z → 97-122
 Space → 32
 0-9 → 48-57

Q. Find longest common prefix string amongst an array of string

Input: strs = ["Flower", "Flow", "Flight"]
 Output: "Fl"

→ If character doesn't match
 → If one string ends

0 1 2 3 4
 apple
 ↑ ↑ ↑
 apps
 ↑ ↑
 ape
 ↑ ↑

count = 0 + 2

TC → O (NO. OF string * length (shortest string))
 Space → O(1)

string: LongestCommonPrefix (vector<string> &strs)
 int n = strs.size();
 string res;
 int min = strs[0].size();

For (int i = 0; i < n; i++)

IF (min > strs[i].size())
 min = strs[i].size();

579
376
235

```

For (int j=0; j<haystack.size(); j++)
{
    For (int i=0; i<needle.size(); i++)
    {
        IF (haystack[j+i] != needle[i])
        {
            return -1;
        }
    }
    res = j;
    return res;
}

```

Q. Implement strstr

Return the index of First occurrence of needle in haystack, or -1, if needle is not part of haystack
 Input: haystack = "hello", needle = "ll", haystack = "aaaa", needle = "bba"
 Output: 2
 Output: -1

haystack = " ", needle = " "

Output: 0

```

int strstr (string haystack, string needle) {
    if (needle.size() == 0) return 0;
    int i=0, j=0, n=haystack.size(), m=needle.size();
    while (i<n)
    {
        j=0;
        if (haystack[i+j] == needle[j])
        {
            int store=i, flag=0;

```

```

while (i < n1 && i < n2)
{
    if (haystack[i] != needle[i])
    {
        flag++;
        break;
    }
    i++;
    i++;
    if (i > n1 && i == n2)
    {
        return store;
    }
    if (i > n1 && flag == 0)
    {
        return store;
    }
    i = store;
    i++;
    return -1;
}

```

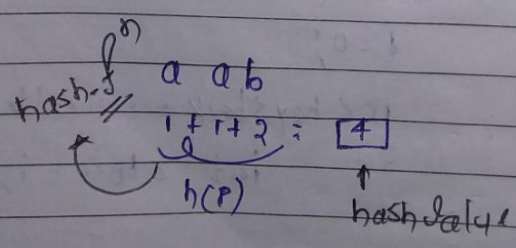
$O(m \times n)$ → size of haystack
size of needle

↓
Optimize -

Rabin-Karp algorithm

Text :- $m=6$ a a a a a b

Pattern :- $m=3$ a a b



Let, Guidence

- a $\rightarrow$ 1
- b $\rightarrow$ 2
- c $\rightarrow$ 3
- d $\rightarrow$ 4
- .
- .
- j $\rightarrow$ 10

sliding window size = 3

Test: a a a a a b
 $1+1+1=3$ $1+1+2=4$

→ Not matching with hash value

Pattern: aab

Draw-back:-

Test: cc a cca a e d b a
 $3+3+1=7$ 7 7

Pattern: d b a
 $m=3$ $1+2+1=7$

Search pattern in every window
 [spurious hits]

To avoid it make strong hash fn

Test: c c a cca a e d b a
 $m=3$

$$3 \times 10^2 + 3 \times 10^1 + 1 \times 10^0 = 300 + 30 + 1 = 331$$

Pattern: d b a

$$m=3 \quad 4 \times 10^2 + 2 \times 10^1 + 1 \times 10^0$$

→ not matching
 reducing spurious hit

because of weak hash fn

see carefully for relation

$$[3 \times 10^2 + 3 \times 10^1 + 1 \times 10^0 - 3 \times 10^2] \times 10 + 3 \times 10^0$$

$$P[i] \times 10^{m-1} + P[i+1] \times 10^{m-2} + P[i+2] \times 10^{m-3}$$

*** To avoid overflow perform modulo

you can change base value according to your convenience, basically you have to design such hash fn so that less spurious hit occurs

$$O(n-m+1)$$

Teacher's Signature.....

Code:-

```
int hash-fn (string s, int m)
{
    int hash-vl = 0;
    for (int i = 0; i < m; i++)
    {
        hash-vl = (hash-vl + s[i]) *
    }
    return hash-vl;
}
```

```
int strStr (string haystack, string needle)
{
    int m = needle.size(), n = haystack.size(), i = 0, j = 0;
    if (m == 0) return 0;
}
```

```
int hash-vl = hash-fn (needle, m);
while (i < n - m + 1)
{
    sum = (sum + haystack[i]) *
    }
    i++;
}
```

```
while (j < n)
```

```
{
    if (sum == hash-vl)
    {
        int k = i, pr = 0;
        while (k <= i)
    }
}
```

Ashwini sir marks

```
ts+1 = (121 (ts - T[st+0h]) + T[st+m+1]) mod q
if (haystack[k] != needle[pr])
{
    break;
}
```

• $h = 121 \text{ mod } q$

```
k++;
pr++;
if (k > j) return i;
}
```

```
j++;
```

```
sum = (sum + haystack[i]) *
sum = sum - haystack[i];
i++;
```

```
return -1;
}
```

Teacher's Signature

QT

reFix F

123456789101112131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869707172737475767778798081828384858687888990919293949596979899100

0 0 1
0 1 2

5 [b a]

λ [0 0]
1 0 1

$O(n^2)$

1 2 3 4 5 6 7 8 9, K=4



Date _____
Page _____

1679

876

235

QT

Prefix Function + (Z) / Z-Function

ba ba ba c b ba

π [0] [0] [1] [2] [3] [4] [0] [1] [1] [2]
0 1 2 3 4 5 6 7 8 9

$b = b$
 $ba \neq ab$

* $\pi(9-1) \rightarrow$ gives longest prefix which matches the longest suffix

defⁿ = length of longest proper prefix starting ending at i not including i

One Question is on GFG

Naive approach

b a b a b a c b b a

π [0] [0] [1] [2] [3] [4] [0] [1] [1] [2]
0 1 2 3 4 5 6 7 8 9

$l = 5, 4, 3 \rightarrow$ at Particular i check for length starting from greater

For (int i=0; i<n; i++)

For (int j=i; j>0; j--)

if (s.substr(0, j) == s.substr(i-j+1, j))

$\pi[i] = j;$
break;

$O(n^2)$

counted time

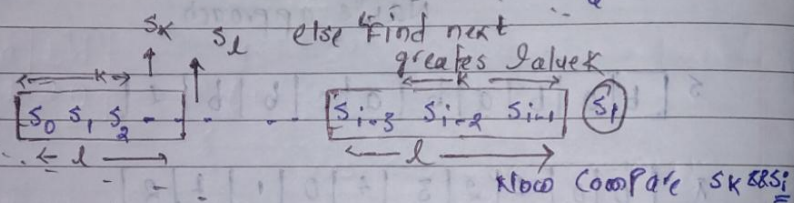
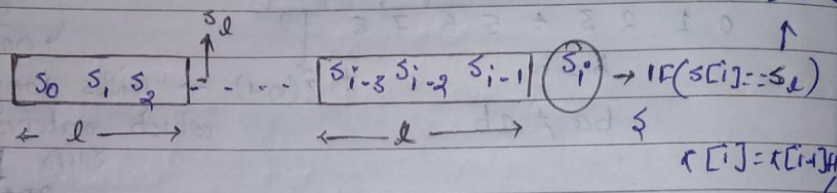
Optimize

π value increases by at most one

s | b | a | b | a | b | a | c | b | b | a |

Property:-
 $\pi[i+1] \leq \pi[i] + 1$

π | 0 | 0 | 1 | 2 | 3 | 4 | 0 | 1 | 1 | 2 |



$k = \pi[l-1]$

so 0 0

$\pi[0] = 0$

For (i = 1 to n-1) {
 $l = \pi[i-1]$ longest suffix possible ending at i-1
 while ($l > 0$ && $s[i] \neq s[l]$)
 {
 $l = \pi[l-1]$
 }
 if ($s[i] == s[l]$)
 {
 $++l$
 $\pi[i] = l$
 }
}

$O(n)$

KMP Algo

s → b a

t → a

q →

q → a

M-1

a	c
0	0

$O(n+m)$

Space →

M-2

s →
(n)

t →
(m)

increase
y at most
one

1 2 2 4 5 6 7 8 9, K=4



Date: / /
Page: /

1679

876

235

KMP Algorithm

s → b a a c a b a c a b a c a b b

t → a c a b a c a

S[TC[i]]

↑

J := S_L

J = T[i+J]

q → [] # []
t s

q → a c a b a c a # b a a c a b - -

Now Find Prefix fⁿ & work is over

whenever get
return value

a c a b a c a # b a a c a b - -
0 0 1 0 1 2 3 0 1 2 3

O(n+m)

Space → O(n+m)

it works because
Prefix value of right
side wouldn't be
greater than

optimize to O(n)

M2:
s → b a a c a b a c a b a c a b b
(n) i-1 i

t → a c a b a c a #
(m) 0 0 1 0 1 2 5 0

count

time

space

3

while (i < n) {

if (text[i] != Pat[j]) {

if (j != 0) {

j = P[j-1];

else

{ i++;

}

else

{ i++;

j++;

if (j == m)

{

res = Push-back(i-m+1);

j = P[j-1];

return res;

backTrack (in naive solⁿ)

a b a b c a b c a b a b a b d

a b a b d

in naive solⁿ, you need to backtrack but here not in KMP algorithm

* you have done this
mistake on GFG

see by tracing

String: a b a b c a b a b a b d

Pattern	a	b	a	b	d
	0	0	1	2	0

Q. Min

Input

0/P

#

Q. La

S =

Q. Minimum character required to make string Palindrome

Input: "ABC"

O/P: 2

Input: "AAACECAAAA"

O/P: 2

$O(n^2)$ Brute Force

Find prefix which is palindrome
↓
return n - length of Palindrome

Optimize

Find length of Prefix which is Palindrome
by using Pi/Z Function

Q. Valid Anagram

s = "anagram", t = "nagaram" s = "rat", t = "car"

true

false

bool isAnagram(string s, string t)

if (s.size() != t.size()) return false;

int i = 0; map<char, int> m1;

for (i = 0; i < s.size(); i++)

{

m1[s[i]]++;

m1[t[i]]--;

}

for (int i = 0; i < m1.size(); i++)

{

if (m1[i] != 0)

{ return false;

}

}

return true;

}



Date

Page

Q. Find all anagrams in a string

I/P:- $s = \text{"cbaebabacd"}$, $P = \text{"abc"}$

O/P: $[0, 6]$

I/P:- $s = \text{"abab"}$, $P = \text{"ab"}$

O/P: $[0, 1, 2]$

Brute Force

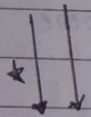


$O(n^2)$ ← check every window of size

Optimized



use sliding window & hash function



Minimum window substring

Given two strings s & t of lengths m & n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no substring, return the empty string $""$.

$s = \text{"ADORECODE BANC"}$, $t = \text{"ABC"}$

O/P = "BANC"





Date _____
Page _____

OF SIR (a)

hash function

o x o
in close
harack
in the
he copy

8.9
8.9
8.9
→ 5
→ 6

fore re

2
↓
agk

1 2
4]

IF we
come
sis

]

10

+

10

10

10

10

10

10

10

10

10

10

inplace → 1 2 3 4 5 6 7 8 9, K=4
 1 2 3 4 5 6 7 8 9 → 5 4 3 2 1 6 7 8 9
 5 4 3 2 1 6 7 8 9 → 5 4 3 2 1 9 8 7 6
 5 4 3 2 1 9 8 7 6 → 6 7 8 9 1 2 3 4

1. Rotate Anticlockwise

5 2
 1 2 3 4 5 → 3 4 5 1 2

Tricks →

To handle large D values, before rotating use

IF (D > N)

D% = N

ex:- temp = arr[0];

For (i=0; i < n-1; i++)

{ arr[i] = arr[i+1];

arr[i] = temp;

For left rotate →

arr[i] = arr[(i+D)*N]

1 2 3 4 5

↓
 anticlockwise

For right rotate →

arr[i] = arr[(i+N-D)*N]

1 2 3 4 5

↓
 clockwise

[we can do (i-D)*N but if we are at 1 then index become -ve so Forth]

2. Union & intersection

using unordered map [coincrement]

similar to hashing

3. Count pair with given sum

unordered_map <int, int> m;

For (int i=0; i < n; i++)

{ m[arr[i]]++;

int twice-count=0;

For (int i=0; i < n; i++)

{ twice-count += m[sum - arr[i]];

if (sum - arr[i] == arr[i])

twice-count--;

Count4
 twice-count
 return
 twice-count
 3

elements are from 0 to $N-1$

4. Find duplicates in an array $\rightarrow$ without using extra array

$arr[] = \{2, 3, 1, 2, 3\}$ Output $\rightarrow 2, 3$

1. Go to index $arr[i] \% n$ and increment its value by 1

2. Now traverse the array again & print all those indexes i for which $arr[i] / n$ is greater than 1.

5. Alternating positive & negative elements with $O(1)$ extra space

$\rightarrow$

Without mentioning order of elements

M-1. $arr[] = \{-2, 3, 4, -1\}$

O/P $= \{-2, 3, -1, 4\} / \{-1, 3, -2, 4\}$

Shift all negative element on left side & positive on right side by using Quicksort approach by taking 0 as pivot

$-2, 3, 4, -1 \rightarrow -2 -1 3 4$

By maintaining order of element

I/P $\rightarrow 1, 2, 3, -4, -1, 4$

O/P $\rightarrow -4, 1, -1, 2, 3, 4$

I/P $\rightarrow -5, -2, 5, 2, 4, 7, 1, 8, 0, -8$

O/P $\rightarrow -5, 5, -2, 2, -8, 4, 7, 1, 8, 0$

- ① negative at odd index
- ② Positive & at even index

↑

First Find out of place element the Find First element after it with opposite sign. we right rotate the subarray b/w these two elements (including these two)

while (i < n && i % 2 == 0)

while (i % 2 == 0 && A[i] < 0 || i % 2 == 1 && A[i] > 0)

{

i++;

i = i+1;

right rotate

see previous page

IF (A[i] > 0)

{

while (A[j] > 0 && j < n)

{

j++;

}

IF (A[i] < 0)

{

while (A[j] < 0 && j < n)

{

j++;

}

IF (j < n)

right rotate (A, i, j, n)

{

i++;

}

Find

$] = \{ 2, \dots$

to in

Now E

ose in

ater

Alterna

th 0

Withou

arr

O/P :

Ft a

de &

sing.

idgt

-2,

By

1, 2, use

→ -4, school

mathematics

6. Find if there is any subarray with sum equal to zero.

I/P → 4, 2, -3, 1, 6

I/P → 4, 2, 0, 1, 6

O/P → Yes

O/P → Yes

→ For (int i=0; i<n; i++)

sum += arr[i];

If current sum has been seen before, then there is a zero sum array. Use hashing to store the sum values.

Maximum subarray sum

7. Kadane's Algorithm → Contiguous subarray with maximum sum

I/P → 1, 2, 3, -2, 5

I/P → -1, -2, -3, -4

O/P → 9

O/P → -1

→ int sum=0, max=-82764

For (int i=0; i<n; i++)

sum += arr[i];

always keep this IF above
below IF, otherwise

some test case will not pass
Ex: [-1]
O/P: 0
Expected: -1

return max;

8. Factorial of a large No.

1. Initialize carry as 0.

2. Do Following For i=0 to res_size-1

(a) Find value of res[i] * x + carry

(b) Update res[i] by storing last digit of Prod in it

(c) Update carry by storing remaining digits in carry

9. Maximum

9

keep

lesser E

1. IF

2

3

4

5

6

7

8

9

10

Best Time

11

12

13

14

15

Do

From

Ex: [7

[

9. Maximum product subarray

→ 7 10 9 0 +8 -1 +2 -2 6 → 24

Keep Max always greater than 1 & Min always lesser than 1.

1. IF $arr[i] > 0$ then Max is $Max * arr[i]$, & Min is $\min(Min * arr[i], 1)$

2. $arr[i] == 0$ Max = 1, Min = 1

3. IF $arr[i] < 0$ then Max is $\max(Min * arr[i], 1)$ & Min becomes $Max * arr[i]$

return Max. Overall max

obtained by every iteration

Best Time to Buy and sell stock

→ Similar intuition like [Done by self] Kadane's algorithm

Ex: [7, 1, 5, 3, 6, 4] O/P: Profit = 6-1 = 5

[7, 6, 4, 3, 1] O/P: - 0

Do its next version 1, 2, 3 also

From leetcode

Ex: [7, 1, 5, 3, 6, 4] O/P: Profit = (5-1) + (6-5) = 7

[1, 2, 3, 4, 5] O/P: - 4

Find c

Next permutation

2, 2, 3

to indi

534976 → 534976 → 536974

→ 536479

Q1

Three pointer algorithm

sort 2, 0, 2, 1, 1, 0 in One pass using O(1)

extra space → Inplace sorting should be

0 0 0 0 1 1 1 ? ? ? 2 2 2

Case 0 → swap (low++, mid++)

Case 1 → mid++

Case 2 → mid, hi--

1. All element be for low 0
2. b/w low & mid 1
3. After high 2
4. Set low and mid at 0 and high at n

1. Max^{ind} index

5. IF equal to 0 swap with low & low++ mid IF 2 swap low high & high-- else just increment mid

→ 132

123

132

213

231

312

321

Brute Force approach → Find all Permutations & choose 132

प्रश्न

Algorithm:-

1 3 5 4 2

→ 1 4 2 3 5

(i) $a[i] < a[i+1]$ $ind1 = i$

(ii) $a[ind2] > a[ind1]$ $ind2 = 3$

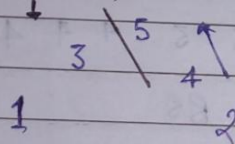
(iii) swap ($a[ind1], a[ind2]$)

1 4 5 3 2

(iv) reverse ($ind1+1 \rightarrow last$)

1 4 2 3 5

intuition $\rightarrow$ not sort



1 3 1
2

→ always

(15) → 14

1 3 5 4 2

1 4 5 3 2 → 1 4 2 3 5

see by writing all permutation

Perm (int A)

if (A == null || A.length <= 1) return;

int i = A.length - 2;

while (i >= 0 && A[i] > A[i+1]) i--;

if (i >= 0)

int j = A.length - 1;

while (A[i] < A[j]) j--;

swap (A, i, j);

{

reverse (A, i+1, A.length - 1);

}

{

Find c

Q. 4-sum Problem

nums = [1, 0, -1, 0, -2, 2]

target = 0

Output: [[-2, -1, 1, 2], [-2, 0, 0, 2], [-1, 0, 0, 1]]

nums = [2, 2, 2, 2, 2], target = 8

Output: [[2, 2, 2, 2]]

You can return answer in any order

4 3 3 4 2 1 2 1 1

1 1 1 2 2 3 3 4 4 4

Sort → 3pt + Bs

target = 9

1 1 1 2 2 3 3 4 4 4

do a binary search on right half

i = 0
j = i + 1
k = i + 1

$\frac{N^3}{\log N} + N \log N$ (sorting)

Optimize it

i j left → 2 2 3 3 4 4 4
right ← 4 4 4

Note: Avoid duplicate (Temp duplicates)



77

$$8 \div 10$$

$$\begin{bmatrix} 5 & 8 & 12 & 1 & 01 & + & 001 \end{bmatrix}$$

$$501 \quad 101 \quad 001 \quad 1 \quad 8 \quad 1 \quad 001$$

in 6 (Major 10s)

with 101 100 101

11 steps 4x2 of 8

$$201 \times 101$$

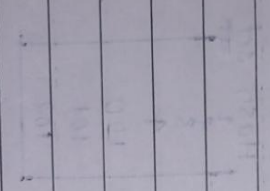
$$\begin{bmatrix} 2 & 8 & 101 & 1 & 001 & 1 & 001 \end{bmatrix}$$

101

$$1011 \text{ yos ab 301s } 101$$

$$8 \times 11$$

right
 $gct = 9$



$$8 \times 11$$

$$101 \times 101$$

2

$$(11)0 + (11)0 = 01$$

$$101 \times 101$$

4b

$$101 \times 101$$

100F
uplucks

$$101 \times 101$$

$$101 \times 101$$

$$101$$

4. Find Q. longest consecutive sequence

ex: $[0, 3, 7, 2, 5, 8, 4, 6, 0, 1]$ ans: $[0, 1, 2, 3]$
 O/P: 9

→

102	4	100	1	101	3	2
-----	---	-----	---	-----	---	---

Sort → 1 2 3 4 5 6 100 101 102

** be careful while O(N log N) + N
 00 11 2 3 4 (corner case)
 2 is such situation

Alte with optimize

102	4	100	1	101	3	2
-----	---	-----	---	-----	---	---

check 102 Not do anything
 4 3

Hash set

100
101
102

shift

side

use

pid

1	2	3	4
---	---	---	---

101 100 exist

3 2 exist

2 1 exist

TC → $O(N) + O(N) + O(N) \log N$

For checking in hash
 check by using 5 4 3 2 1

Q. largest

unordered

int sy

if not

-1 -1 -1

O/P = 2

Expect

Q1 (100)

Count

Given

Count

there

Input

Output

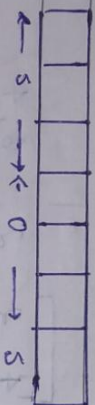
Expla

element

Q. Largest subarray with 0 sum.

1, -1, 3, 2, -2, -8, 1, 7, 10, 28

notations



unordered - map <int, int> mp For (i=0; i < n; i++)

int sum = 0, i = 0, size = 0; if (sum != 0) size = i + 1

if not written you always forget to write

-1 1 -1 1

O/P = 2

Expected: 4

if (mp.size() != 0 & mp.find(sum) != mp.end())

if (size < (i - mp[sum]))

size = i - mp[sum]

return size;

mp[sum] = i;

Q1 (imp)

Count the number of subarrays having a given XOR.

Given an array of integers arr[] and a number Count the number of subarrays having XOR of their element as x.

Input: arr[] = {4, 2, 2, 6, 4, 6}

Output: 4

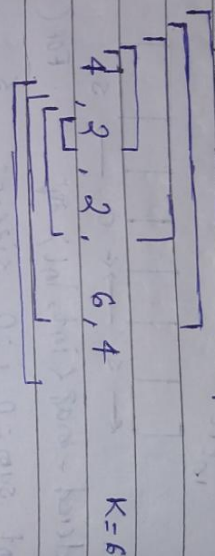
Explanation: The subarrays having XOR of their element as 6 are {4, 2, 2}, {4, 2, 2, 6}, {4, 2, 2, 6, 4}, {4, 2, 2, 6, 4, 6}

Q. Input: arr [] = {5, 6, 7, 8, 9}, m = 5

Output: 3

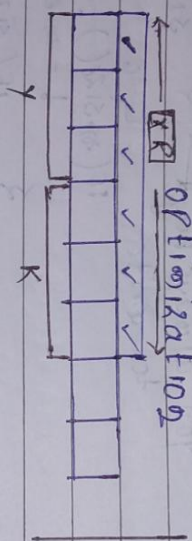
Explanation: The subarrays having XOR of their elements as 5 are {5} and {5, 6, 7, 8, 9}

Brute Force:



$O(m^2)$

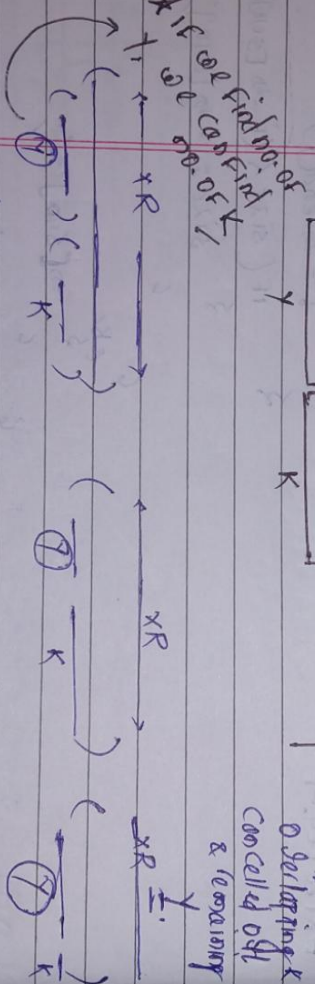
Optimization



$Y \times K = X \times R$

$Y = X \times R \times X$

0 delagings & cancelling off



* there can be multiple For given index

XOR = 0

Cont = 0 & 4

fixed [4, 2, 2, 6, 4]

XOR = 4 ^ 2 = 6 ^ 2

= 4 ^ 6 = 2

2 ^ 4 = 6

Y = 2 ^ 6

= 4

(6, 1)
(4, 2)

freq(hash)

TC $\rightarrow O(N \log N)$
SC $\rightarrow O(N)$

Kac

Input: [-]
Output: [-]

Max

int solve (arr A, int B) {

if (arr < int, int) {
int cnt = 0, xor = 0;

for (auto it : A) {

xor = xor ^ it;

if (xor == B) {

cnt++;
if (cnt == 1) {

return cnt;

cnt = find (xor ^ B);

return cnt + 1;

}

Kadane's Algorithm

Maximum subarray sum

Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

Output: 6 [4, -1, 2, 1]

Brute Force

$O(n^3)$ → Find all subarray

Optimal

sum = 0

max = arr[0]

[-2, -3, 4, -1, -2, 1, 5, 3]

9, 4, 12

int maxSubArray (vector<int> &nums) {

int sum = 0;

int maxi = INT_MIN;

for (auto it : nums) {

sum += it;

maxi = max (sum, maxi);

if (sum < 0) sum = 0;

return maxi;

#Q. Maximum Product subarray

Brute Force

$O(n^3)$

↓
Optimise

a → [-1, 6, 2, 0, 7, 9] → 6

we have to handle just three cases

Case-1 Current element is a +ve integer

-1, 6, 2, 3

Pref-max = Prod = 12

curr-max-Prod = 12 * 3 = 36

Case 2 → current element is -ve integer

-1, 6, 2, -2

Pref-max = 12, Pref-min-Prod = 12

curr-max-Prod = -12 * -2 = 24

template
ibray
doesn't
find max
of three
Use this

Case-3

Current element is starting pos

-1, 6, 2, 0, 7, 9

↑
current

$$\text{curr-max-product} = \text{max}(\text{prev-max-product} * a[i], \text{prev-min-product} * a[i])$$

$$\text{curr-min-product} = \text{min}(\text{prev-max-product} * a[i], \text{prev-min-product} * a[i])$$

def max-product-subarray(a):

$$\text{curr-max-product} = a[0]$$

$$\text{prev-max-product} = a[0]$$

$$\text{prev-min-product} = a[0]$$

$$\text{curr-min-product} = a[0]$$

$$\text{ans} = a[0]$$

for i in range(1, len(a)):

$$\text{temp-max} = \text{curr-max-product} = \text{max}(\text{prev-max-product} * a[i], \text{prev-min-product} * a[i])$$

$$\text{temp-min} = \text{curr-min-product} = \text{min}(\text{prev-max-product} * a[i], \text{prev-min-product} * a[i])$$

$$\text{curr-max-product} = \text{temp-max}$$

$$\text{curr-min-product} = \text{temp-min}$$

$$\text{ans} = \text{max}(\text{ans}, \text{curr-max-product})$$

$$\text{prev-max-product} = \text{curr-max-product}$$

$$\text{prev-min-product} = \text{curr-min-product}$$

return ans

Just dry run -1, 6, 2, 0, 7, 9 to take

the feel

curr-max-product = a

Q. Repeat and Missing Number array

Each integer appears exactly one except
A which appears twice and B which is
missing Return A and B:

Input: [3, 1, 2, 5, 3]

constraint $\Rightarrow O(n)$

Output: [3, 4]

$$O(n), O(n) + \text{missing}$$

$\Downarrow$

$$1 + 2 + 3 + \dots + n = n \times (n+1) / 2 = \sum$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = n \times (n+1) \times (2n+1) / 6 = \sum$$

$x \rightarrow \text{missing}$, $y \rightarrow \text{repeating}$

$x - y = \sum - S$ given array sum

$$x^2 - y^2 = \sum^2 - S^2$$

as order of occurrence of
each element is just 1

using XOR

ex:

4	3	6	2	1	1
---	---	---	---	---	---

$x \oplus x = 0$

$$4 \wedge 3 \wedge 6 \wedge 2 \wedge 1 \wedge 1 = 8$$

$\sum$
Nao
light

vector

key

One except
8 which is

$$3 \wedge (1 \wedge 2 \wedge 3 \wedge 4 \wedge 5 \wedge 6) = 4$$

*

$$x \wedge y = 4 \quad - \textcircled{1}$$

$$(1 \wedge 3 \wedge 6 \wedge 2 \wedge 1 \wedge 1) \wedge (1 \wedge 2 \wedge 3 \wedge 4 \wedge 5 \wedge 6)$$

Now use separation

$$x \oplus y = 1 \oplus 0 \oplus 0 \rightarrow 1$$

$$y = 10$$

$$\begin{bmatrix} 6 \\ 4 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$$

$$2, 1, 2, 3, 4, 5, 6, 6$$

$$5, 1$$

$$\begin{bmatrix} 6 \\ 5 \\ 4 \end{bmatrix}$$

$$\begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix}$$

array 5400

45 05 54 05

00 13 54 05
being

vector < int > solution (

$$\text{int } x0, 1$$

$$\text{int } \text{set-bit} = 00;$$

$$\text{int } i = 0;$$

$$\text{int } x = 0;$$

$$\text{int } y = 0;$$

$$\text{int } n = \text{arr.size()};$$

$$x0, 1 = \text{arr}[0];$$

$$11 = 3$$

```
for (i = 1; i < n; i++)
    x0r1 = x0r1 ^ arr[i];
```

```
for (i = 1; i < n; i++)
    x0r1 = x0r1 ^ i;
```

set-bit-no = x0r1 & ~(x0r1-1)

```
for (i = 0; i < n; i++) {
    if (arr[i] & set-bit-no)
```

$x = x \wedge arr[i];$

else

$y = y \wedge arr[i];$

}

```
for (i = 1; i < n; i++) {
```

$\text{if } (i \& \text{set-bit-no})$

$x = x \wedge i;$

else

$y = y \wedge i;$

}

```
int x-count = 0;
```

```
for (int i = 0; i < n; i++) {
```

$\text{if } (arr[i] == x)$

$x-count++;$

$\text{if } (x-count == 0)$

return &y, &x;

return &y;

Diagona

an

it zero

so,

no of

5 = 10 bytes

4	3	2	1
0	1		